

Formal Modeling and Security Analysis for OpenFlow-based Networks

Yongxin Zhao^{1,2}, Xi Wu^{3*}, Jing Liu¹ and Yilong Yang⁴

¹Shanghai Key Laboratory of Trustworthy Computing,
East China Normal University, Shanghai, China

²Shanghai Trusted Industrial Control Platform Co., Ltd., China

³School of ITEE, the University of Queensland, Brisbane, Australia

⁴Computer Information and Science, Faculty of Science and Technology,
University of Macau, Macau, China

Abstract—We present a formal OpenFlow-based network programming language (OF) including various flow rules, which can not only describe the behaviors of an individual switch, but also support to model a network of switches connected in the point-to-point topology. Besides, a topology-oriented operational semantics of the proposed language is explored to specify how the packet is processed and delivered in the OpenFlow-based networks. Based on the formal framework, we also propose an approach to detect potential security threats caused by the conflict of dynamic flow rules imposed by dynamic OpenFlow applications.

Index Terms—OpenFlow-based Networks, Operational Semantics, Security Analysis

I. INTRODUCTION

Recently, a new paradigm, Software-Defined Networking (SDN) [1], has been proposed to use a programmable network infrastructure to separate the control plane and the data plane. As the current state-of-the-art network architecture, SDN introduces the benefits of a centralized approach to network management and configuration. Since OpenFlow is the most deployed SDN concept, in this paper, we focus on the OpenFlow-based networks. Many researchers have paid attention to the security issues of the OpenFlow-based networks and proposed several approaches [2]–[5] to deal with the issues. However, these approaches are lack of a trustworthy semantic foundation to demonstrate their correctness and they may also cause a false alarm in some cases (e.g., [5]); that is, a conflict detected by the algorithm, in actual, would not arise at all. Thus developing a formal approach to detect the conflict among dynamic flow rules is still challenging.

Anderson et al. presented a network programming language NetKAT [6] to describe and reason about the behaviors of the network programs based on Kleene Algebra with Tests (KAT). Inspired by the related work [5] [6], in this paper, we propose a formal framework to reason about the correctness of the behaviors of the flow rules and to detect potential conflicts among flow rules imposed by dynamic OpenFlow applications. Firstly, we present a formal network programming language OF including various flow rules to describe the behaviors of the OpenFlow-based networks, which incorporate switches in

the point-to-point topology. Besides, an operational semantics of our language is explored to specify how the packet is processed for network programs. Based on the framework, we also propose a detecting approach to check the potential contradictions of flow rules. The correctness of our approach is established by the formal semantic foundation.

The remainder of this paper is organized as follows. Section II gives a brief introduction to the OpenFlow architecture. We present our OF language and its topology-oriented operational semantics in Section III. Section IV proposes a detecting approach to check conflicts among flow rules and demonstrates the effect of our approach by a case study. Section V concludes our achievements and indicates further directions.

II. OPENFLOW ARCHITECTURE

The OpenFlow architecture [7] shown in Fig. 1 includes three components: Switch, Controller and OpenFlow protocol.

A switch is composed of one group table, multiple flow tables, and an OpenFlow channel. A flow table is composed of several flow entries, each of which includes at least one simple action on that flow item. In this paper, we mainly focus on the source and destination addresses as well as the forwarding, dropping and updating instructions. For example, one flow entry can update the destination address of a packet into c if the packet is delivered from a to b .

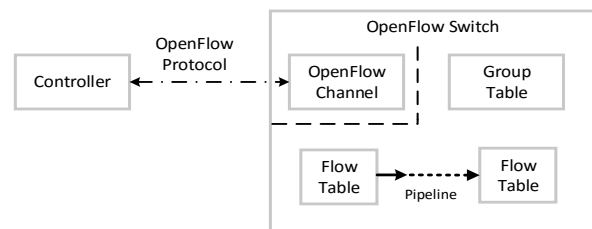


Fig. 1. The OpenFlow Architecture

Controllers are designed to manage a collection of switches for traffic control. A controller can deploy, update or delete flow entry rules from a flow table on behalf of the administrator. OpenFlow protocols are flow-oriented protocols to

* Corresponding author: xi.wu@uq.edu.au

control the flow, through which an OpenFlow controller can communicate with the switches.

III. OPENFLOW-BASED NETWORK PROGRAMMING LANGUAGE

In this section, we present some additional definitions and propose our OF language together with the topology-oriented operational semantics to describe and specify how the packet is processed in network programs.

A. Packet and Network Topology

The packet refers to the basic transmission unit on the network layer, which is constructed of two major pieces: the packet header and the data. In our framework, we don't consider the data part of a packet since it is independent of the packet routing process. Also, instead of using concrete addresses such as IP addresses and TCP/UDP ports, we only introduce abstract addresses to represent the address information of packets.

Definition 3.1 (Packet) A packet pk is defined as a 4-tuple (src, dst, typ, loc) , consisting of

- the source address src from which the packet is sent;
- the destination address dst to which the packet is sent;
- the protocol type typ of the packet;
- the current address loc of the packet.

Given a tuple t , we define the projections $\pi_i(t)$ ($i = 1, 2, \dots, |t|$) to select the i -th component of t and $|t|$ stands for the number of the tuple componets. We use notation $pk[e/f]$ to represent a new packet obtained from pk by updating f to e . Besides, we also introduce two particular packets for semantics definitions, i.e., null packet ϕ and fault packet $\perp$. The former one indicates the packet has been discarded during the transmission, while the latter one represents the status of the packet is unknown. The notation PK denotes the set of all the packets.

Obviously, a network topology could be regarded as an undirected graph with communicating devices as nodes, and connections as edges between the nodes. For simplicity, we identify nodes with their corresponding addresses in our model. Thus a network topology T is denoted as a tuple (V, E) , where V is the set of all communicating devices and $E \subseteq V \times V$ is the set of all connections in the network.

B. OF Language

The syntax of our OF language can be found in Table I, which could be used to formally describe the flow tables configured in all switches in the OpenFlow-based networks.

In the syntax, e represents an expression. Specially, we use the notation b to stand for a boolean expression defined over the set of fields of the packet. We use FR to denote the set of all flow rules. The flow rules are interpreted as follows.

- Rule Error indicates that the execution of the flow rule encounters an error within the topology. Rule Skip states that the flow rule makes no change to the status of the packet. Rule Drop represents the flow rule discards all the

TABLE I
SYNTAX

Flow Rules	p, q	$::=$	error	Error
			skip	Skip
			drop	Drop
			$[b]$	Filter
			$src :=_s e$	UpSrc
			$dst :=_d e$	UpDst
			$loc :=_l e$	Forward
			$p; q$	Sequential
			$p + q$	Choice
			if b then p else q	Conditional
			$[while\ b\ do\ p]_n$	Bounded Iteration

input packets. Rule Filter $[b]$ indicates that the packets, which do not satisfy the boolean expression b would be discarded and the others would keep unchange.

- Rules UpSrc $src :=_s e$ and UpDst $dst :=_d e$ update the fields src and dst of the packet with e , respectively. Rule Forward $loc :=_l e$ forwards the packet to the address e from its current address. Obviously, the success of the transmission depends on whether the connection between address e and the current address of the packet exists. We assume that the type of the value of the expression e matches the type of the field of the packet accordingly.
- Like traditional programming languages, Sequential $p; q$ implies the execution order of the flow rules, i.e., rule p is executed followed by rule q . Choice $p + q$ refers that flow rules p and q are executed nondeterministically.
- Rule Conditional if b then p else q indicates that rule p is executed if b is satisfied, otherwise rule q is executed.
- Rule Bounded Iteration $[while\ b\ do\ p]_n$ indicates that the iteration would finish when b yields false. Besides, the upper bound of the loop times is limited to n . Here, n is an integer and $n \geq 0$.

C. Topology-oriented Operational Semantics

In this subsection, the topology-oriented operational semantics precisely specifies how the packet is processed in network programs. It is given in terms of transitions with respect to the topology.

Definition 3.2 (Execution state) An execution state is defined as $\langle p, pk \rangle_l$, where

- p is a rule to be executed at address l ;
- pk is a packet to be processed based on rule p ;
- l is a tag representing the address of the state.

The topology-oriented transition is defined from the execution state to the packet with respect to the topology. All the transitions are listed in Table II. (ASen) states that the flow rules would not deal with the packet at other locations. (Null) and (Fault) indicate that neither a null packet nor a fault one would be updated or forwarded by flow rules.

(Skip) describes that rule skip doesn't make any change on the packet pk . (Drop) means rule drop just discards the packet regardless of what the input is and outputs a null packet. (Error) indicates whatever the input is, it will output a fault packet. (Filter1) and (Filter2) are used to select the

TABLE II
THE TOPOLOGY-ORIENTED TRANSITIONS

(ASen) $\frac{l \neq \pi_4(pk)}{T \vdash \langle p, pk \rangle_l \rightarrow pk}$	(Null) $\frac{}{T \vdash \langle p, \phi \rangle_l \rightarrow \phi}$	(Fault) $\frac{}{T \vdash \langle p, \perp \rangle_l \rightarrow \perp}$	(Skip) $\frac{}{T \vdash \langle \text{skip}, pk \rangle_l \rightarrow pk}$
(Drop) $\frac{l = \pi_4(pk)}{T \vdash \langle \text{drop}, pk \rangle_l \rightarrow \phi}$	(Error) $\frac{l = \pi_4(pk)}{T \vdash \langle \text{error}, pk \rangle_l \rightarrow \perp}$	(Filter1) $\frac{l = \pi_4(pk), b(pk) = \text{true}}{T \vdash \langle [b], pk \rangle_l \rightarrow pk}$	(Filter2) $\frac{l = \pi_4(pk), b(pk) = \text{false}}{T \vdash \langle [b], pk \rangle_l \rightarrow \phi}$
(UpSrc) $\frac{l = \pi_4(pk)}{T \vdash \langle \text{src} :=_s e, pk \rangle_l \rightarrow pk[e/\text{src}]}$	(UpDst) $\frac{l = \pi_4(pk)}{T \vdash \langle \text{dst} :=_d e, pk \rangle_l \rightarrow pk[e/\text{dst}]}$	(Fwd1) $\frac{l = \pi_4(pk), (l, e) \in E(T)}{T \vdash \langle \text{loc} :=_l e, pk \rangle_l \rightarrow pk[e/\text{loc}]}$	
(Fwd2) $\frac{l = \pi_4(pk), (l, e) \notin E(T)}{T \vdash \langle \text{loc} :=_l e, pk \rangle_l \rightarrow \perp}$	(Seq) $\frac{T \vdash \langle p, pk \rangle_l \rightarrow pk', T \vdash \langle q, pk' \rangle_l \rightarrow pk''}{T \vdash \langle p; q, pk \rangle_l \rightarrow pk''}$	(Choice1) $\frac{T \vdash \langle p, pk \rangle_l \rightarrow pk'}{T \vdash \langle p + q, pk \rangle_l \rightarrow pk'}$	
(Choice2) $\frac{T \vdash \langle q, pk \rangle_l \rightarrow pk'}{T \vdash \langle p + q, pk \rangle_l \rightarrow pk'}$	(Cond1) $\frac{b(pk) = \text{true}, T \vdash \langle p, pk \rangle_l \rightarrow pk'}{T \vdash \langle \text{if } b \text{ then } p \text{ else } q, pk \rangle_l \rightarrow pk'}$	(Cond2) $\frac{b(pk) = \text{false}, T \vdash \langle q, pk \rangle_l \rightarrow pk'}{T \vdash \langle \text{if } b \text{ then } p \text{ else } q, pk \rangle_l \rightarrow pk'}$	
(Loop1) $\frac{n = 0 \vee b(pk) = \text{false}}{T \vdash \langle [\text{while } b \text{ do } p]_n, pk \rangle_l \rightarrow pk}$	(Loop2) $\frac{b(pk) = \text{true}, n \geq 1, T \vdash \langle p, pk \rangle_l \rightarrow pk', T \vdash \langle [\text{while } b \text{ do } p]_{n-1}, pk' \rangle_l \rightarrow pk''}{T \vdash \langle [\text{while } b \text{ do } p]_n, pk \rangle_l \rightarrow pk''}$		

packets depending on whether b is satisfied. (UpSrc) and (UpDst) update the value of the fields src and dst respectively. (Fwd1) indicates the packet is delivered successfully if the link between addresses l and e exists in Topology T , otherwise a fault packet is produced which is captured by rule (Fwd2).

(Seq) describes the execution of the sequential composition of rule p and q . (Choice1) and (Choice2) represent rule $p + q$ could execute either p or q nondeterministically. (Cond1) indicates if b then p else q would execute p when b is evaluated true, otherwise q is selected to be executed, which is shown by (Cond2). (Loop1) gives the termination conditions of the loop, that is, either n equals 0 or b is evaluated to be false. (Loop2) shows how to compute the output result of the loop when the iteration times are bounded to n .

Next, we introduce a semantics function to give the set of all packets resulted from the rule p at node l , which is defined by the topology-oriented transitions.

Definition 3.3 (Semantics function) Given a topology T and a node l , we define a semantics function $S[p; T, l](pk)$ of the rule p as below: $S[p; T, l](pk) =_{df} \{pk' | T \vdash \langle p, pk \rangle_l \rightarrow pk'\}$

We extend the semantics function from one node to a path, which is a sequence of nodes linked by the connections in the network. Additionally, we introduce a flow table network $RN = (V, E, F)$ by deploying flow rules on each node in the network, where F is a rule function from V to FR . Then, we can define *output packets* to represent the output packets on any given path in a flow table network.

Definition 3.4 (Output packets) Given a flow table network $RN = (V, E, F)$ and a path $\vec{l} = (v_1, \dots, v_n)$, the output packets of pk on $\vec{l}$ is denoted by $Out[RN](\vec{l}, pk)$, where,

$$Out[RN](\vec{l}, pk) =_{df} \{pk' | \exists pk_i, i \in 1..(n-1) \cdot pk_0 = pk \wedge pk_i \in S[F(v_i); (V, E), v_i](pk_{i-1}) \wedge pk' \in S[F(v_n); (V, E), v_n](pk_{n-1})\}$$

IV. SECURITY ANALYSIS AND CASE STUDY

In this section, we propose an approach to detect potential security threats caused by the conflict between flow rules

and firewall's policy. Compared with the approach in [5], our approach would not lead to a false alarm and the correctness of which is established based on our formal semantics.

In general, a firewall is employed to monitor and control the incoming and outgoing packets based on predetermined security rules, which could be described by our OF language. For instance, rule (if ($\text{src} = SRC \wedge \text{dst} = DST$) then drop) means that the firewall would block all packets sent from SRC to DST . Simply, we represent it as $(SRC \rightarrow DST)$ then drop.

One important feature of the OpenFlow-based network is that it allows the source address and the destination address of the packet to be deliberately modified by flow rules deployed in network nodes. Because of the novel feature, some malicious packets delivered by the attackers may bypass the firewall and lead to a security threat. Essentially, it brings a conflict between flow rules and firewall's policy and the firewall would not play a role in protecting the trusted internal network. Therefore, we intend to propose a formal approach to detecting whether the firewall works well in the OpenFlow-based networks. Firstly, we give a definition of conflict between flow rules and firewall's policy.

Definition 4.1 (No Conflict) In a flow table network configured with the firewall's policy $(SRC \rightarrow DST)$ then drop, we say there is no conflict between flow rules and firewall's policy if all packets from the node SRC could not be finally delivered to the node DST .

Based on our formal framework and Definition 4.1, we give a decision theorem to detect the existence of conflict. Our approach is carried out by checking whether the packets from source node could be delivered to the destination one.

Theorem 4.2 (Conflict detecting) No conflict in network RN with respect to fire wall's policy $(SRC \rightarrow DST)$ then drop if and only if $\forall pk \in PK, \forall l \cdot \pi_4(pk) = \pi_1(\vec{l}) = SRC \wedge \pi_{|\vec{l}|}(\vec{l}) = DST \wedge Out[RN](\vec{l}, pk) \setminus \{\perp, \phi\} = \emptyset$.

Here we consider a case study in Fig. 2 to demonstrate how to detect a conflict in the OpenFlow rules base on our

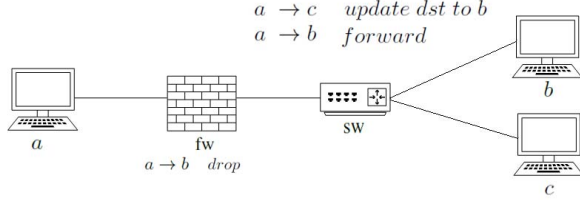


Fig. 2. Case Study

proposed approach. In this case study, there are three hosts (denoted as a , b and c), one OpenFlow switch (represented as sw) and one firewall (represented as fw). The firewall is used to block any packet that will be sent from host a to b . Based on the topology, all the links in the network can be given as:

$$E = \{(a, fw), (fw, sw), (sw, c), (sw, b)\}$$

Step 1. We can compute all paths from host a to b based on a given network topology T as: $a \rightarrow fw \rightarrow sw \rightarrow b$. We just regard this path as a sub-topology, represented as T_1 . Then, we can have:

$$E(T_1) = \{(a, fw), (fw, sw), (sw, b)\}$$

Step 2. For simplicity, we initialize the input packet as $pk = (a, c, SSH, a)$ which means that the packet on host a will be sent to host c , and it is currently located on a .

Step 3. We calculate the output packets of the initial packet pk on path T_1 based on the definition 3.4, and put the results in the set OP , that is $OP = Out[RN](T_1, pk)$. where $RN = (\{a, fw, sw, b\}, E(T_1), F)$. Firstly, we use our OF language to describe the flow rules.

$$\begin{aligned} F(a) : loc &:=_i fw \\ F(fw) : (a \rightarrow b) &\text{ then drop else } (loc :=_i sw) \\ F(sw) : (a \rightarrow c) &\text{ then } (dst :=_d b) \\ &(a \rightarrow b) \text{ then } (loc :=_i b) \end{aligned}$$

According to definition 3.3 and our operational semantics, we obtain the output packets step by step as follows.

$$\frac{a = \pi_4(pk) \wedge (a, fw) \in E(T_1)}{T_1 \vdash \langle loc :=_i fw, pk \rangle_a \rightarrow pk [fw/loc]}$$

let $pk_1 = pk [fw/loc] = (a, c, SSH, fw)$. Then the packet pk_1 is forwarded from fw to sw based on path T_1 .

$$\frac{\pi_2(pk_1) \neq b, fw = \pi_4(pk_1) \wedge (fw, sw) \in E(T_1)}{T_1 \vdash \langle loc :=_i sw, pk_1 \rangle_{fw} \rightarrow pk_1 [sw/loc]}$$

$$\frac{}{T_1 \vdash \langle (a \rightarrow b) \text{ then drop else } (loc :=_i sw), pk_1 \rangle_{fw} \rightarrow pk_2}$$

where $pk_2 = pk_1 [sw/loc] = (a, c, SSH, sw)$. There are two flow rules inside the switch sw and the corresponding semantic transitions are given as follows.

$$\frac{sw = \pi_4(pk_2)}{T_1 \vdash \langle \langle dst :=_d b \rangle, pk_2 \rangle_{sw} \rightarrow pk_2 [b/dst]}$$

$$\frac{(\pi_1(pk_2) = a) \wedge (\pi(pk_2) = c)}{T_1 \vdash \langle (a \rightarrow c) \text{ then } (dst :=_d b), pk_2 \rangle_{sw} \rightarrow pk_2 [b/dst]}$$

let $pk_3 = pk_2 [b/dst] = \{a, b, SSH, sw\}$.

$$\frac{sw = \pi_4(pk_3), (sw, b) \in E(T_1)}{T_1 \vdash \langle \langle loc :=_i b \rangle, pk_3 \rangle_{sw} \rightarrow pk_3 [b/loc]}$$

$$\frac{(\pi_1(pk_3) = a) \wedge (\pi(pk_3) = b)}{T_1 \vdash \langle (a \rightarrow b) \text{ then } (loc :=_i b), pk_3 \rangle_{sw} \rightarrow pk_3 [b/loc]}$$

let $pk_4 = pk_3 [b/loc] = (a, b, SSH, b)$. Finally we can get $OP = \{pk_4\} = \{(a, b, SSH, b)\}$.

Step 4. Evaluate the output packet OP that it contains only one element, however, this element is neither ϕ nor $\perp$. According to Theorem 4.2, we can conclude there exists a conflict on this path to violate the rule of firewall on the current packet.

In summary, because the packet can be modified by flow rules in the OpenFlow networks, there may exist a conflict packet which can bypass the firewall's policy. Applying our approach on cases makes the detection of conflicts more sufficient.

V. CONCLUSION AND FUTURE WORK

In this paper, we present a formal network programming language OF with its formal semantics to model and describe the behaviors of the OpenFlow-based networks, and propose an approach to check the potential security threats caused by the conflict between dynamic flow rules and the firewall's policy. In the future, we will implement our operational semantics in Maude.

ACKNOWLEDGEMENT

This paper is partially supported by National Key Research and Development Project (No. 2017YFB1001800), Science and Technology Commission of Shanghai Municipality Project (No. 18ZR1411600), National Defense Basic Scientific Research Program of China (No. JCKY2016204B503) and NSFC Key Project (No. 61332008) and NSFC (No. 61572195).

REFERENCES

- [1] B. A. A. Nunes, M. Mendonca, X. N. Nguyen, K. Obraczka, and T. Turetli, "A Survey of Software-Defined Networking: Past, Present, and Future of Programmable Networks," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 3, pp. 1617 – 1634, 2014.
- [2] S. Son, S. Shin, V. Yegneswaran, and P. Porras, "Model Checking Invariant Security Properties in OpenFlow," in *IEEE International Conference on Communications*, 2013, pp. 1974 – 1979.
- [3] E. Al-Shaer and S. Al-Haj, "FlowChecker: Configuration Analysis and Verification of Federated OpenFlow Infrastructures," in *In Proc. 3rd ACM Workshop on Assurable and Usable Security Configuration*. ACM, 2010, pp. 37 – 44.
- [4] A. Lara and B. Ramamurthy, "OpenSec: A Framework for Implementing Security Policies using OpenFlow," in *Global Communications Conference*, 2015, pp. 781 – 786.
- [5] P. Porras, S. Shin, V. Yegneswaran, M. Fong, M. Tyson, and G. Gu, "A Security Enforcement Kernel for OpenFlow Networks," in *In Proc. 1st Workshop on Hot topics in Software Defined Networks*. ACM, 2012, pp. 121 – 126.
- [6] C. J. Anderson, N. Foster, A. Guha, J.-B. Jeannin, D. Kozen, C. Schlesinger, and D. Walker, "NetKAT: Semantic Foundations for Networks," *SIGPLAN Not.*, vol. 49, no. 1, pp. 113 – 126, January 2014. [Online]. Available: <http://doi.acm.org/10.1145/2578855.2535862>
- [7] O. N. Foundation, "OpenFlow Switch Specification Version 1.3.4 [White Paper]," 2014.